
EEE 6778. Applied Machine Learning II



Module 5. Review of Topics

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a stepped, architectural effect.

Time to Recap!

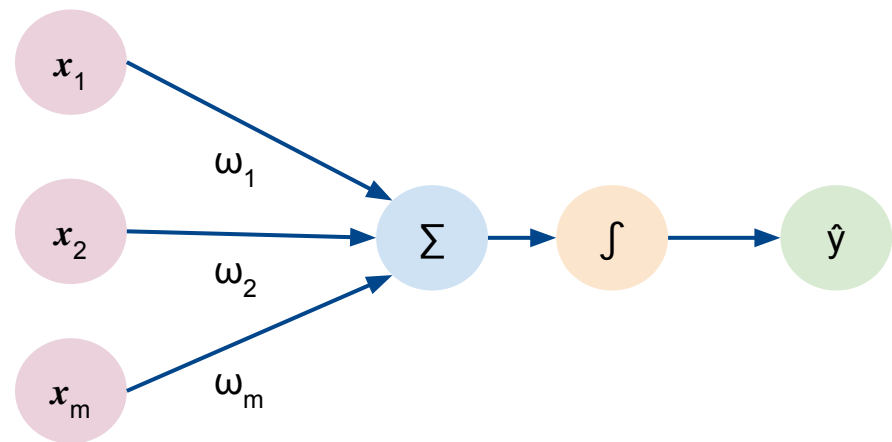
Topics covered

1. Artificial Neural Networks
2. Convolutional Neural Networks
3. Generative Adversarial Networks
4. Autoencoders
5. Recurrent Neural Networks
6. Attention and Natural Language Models

Topics covered

1. **Artificial Neural Networks**
 - a. Perceptron
 - b. Neural Networks built from Perceptrons
 - c. Loss Optimization and Gradient descent
 - d. Backpropagation
 - e. Convergence and Training Optimization
2. Convolutional Neural Networks
3. Generative Adversarial Networks
4. Autoencoders
5. Recurrent Neural Networks
6. Attention and Natural Language Models

The Perceptron: Forward Propagation



Inputs

Weights

Sum

Non-
linearity

Output

Output

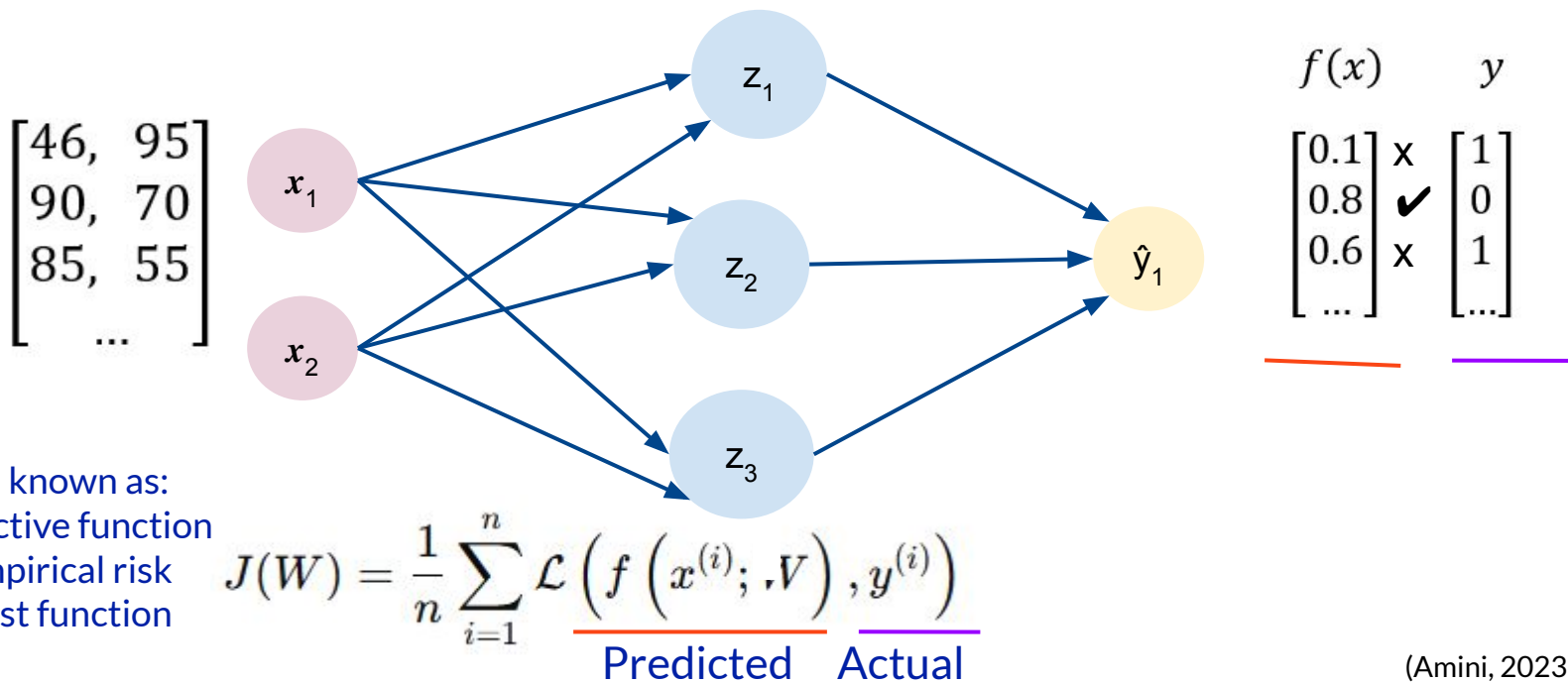
Linear combination
of outputs

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Non-
Linear
Activation
function

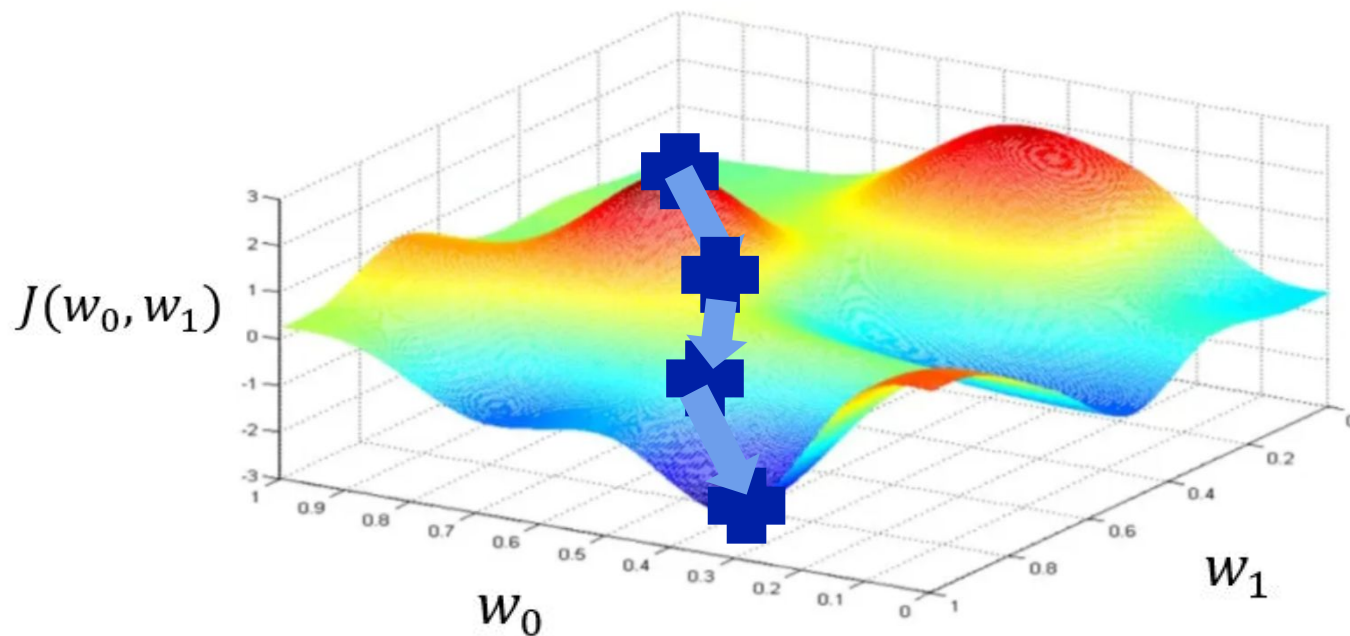
Empirical Loss

The empirical loss measures the total loss over our entire dataset



Gradient Descent

Repeat until convergence



Loss Optimization

We want to find the network weights that achieve the **lowest loss**

$$W^* = \arg \min_W \frac{1}{n} \sum_{i=1}^n \mathcal{L} \left(f \left(x^{(i)}; W \right), y^{(i)} \right)$$

$$W^* = \arg \min_W J(W)$$

Where:

- W^* represents the optimal parameter values that minimize the loss function.
- The first equation expresses that the optimal parameters W are found by minimizing the average loss function over all training samples.
- The second equation simplifies the notation by defining the loss function as $J(W)$, indicating that the goal is to minimize this function to obtain the best model parameters.

Convergence

1. Faster Optimizers
2. Non saturating
3. Normalization
4. Overfitting

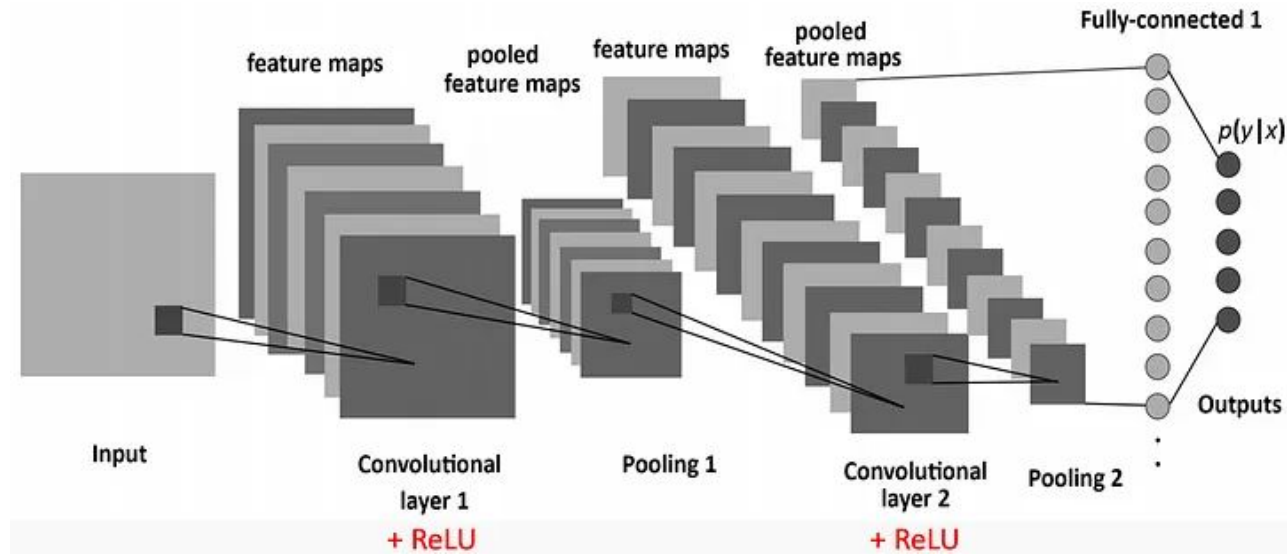
Ways to Optimize Neural Network Training...Amplified!

1. Use efficient optimizers (e.g., AdamW, Adam, etc.).
2. Utilize hardware accelerators (GPUs/TPUs).
3. Max out the batch size.
4. Use Bayesian Optimization if the hyperparameter search space is large.
5. Set `max_workers` in the DataLoader.
6. Set `pin_memory` in the DataLoader.
7. Use mixed precision training.
8. Use He or Xavier initialization for faster convergence.
9. Use activation checkpointing to optimize memory (noting that runtime will increase).
10. Leverage multi-GPU training (Model/Data/Pipeline/Tensor parallelism).
11. For large models, use DeepSpeed, FSDP, or YaFSDP.
12. Normalize data after transferring to the GPU (especially for integer data).
13. Use gradient accumulation (though improvements may be marginal).
14. Always use `DistributedDataParallel`, not `DataParallel`.
15. Create tensors directly on the GPU (e.g., `torch.rand(2,2, device=...)`) rather than creating on CPU and then transferring (e.g., `torch.rand(2,2).cuda()`).

Topics covered

1. Artificial Neural Networks
2. **Convolutional Neural Networks**
 - a. Convolutional Filters
 - b. Visualizing Trained CNNs
 - c. Object Detection
 - d. Image Segmentation
3. Generative Adversarial Networks
4. Autoencoders
5. Recurrent Neural Networks
6. Attention and Natural Language Models

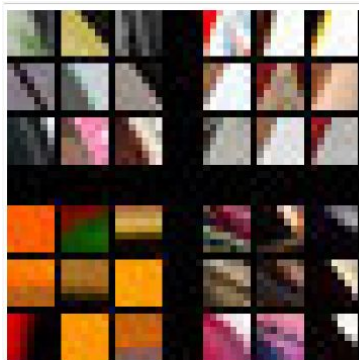
UF Convolutional Filters - Feature Maps



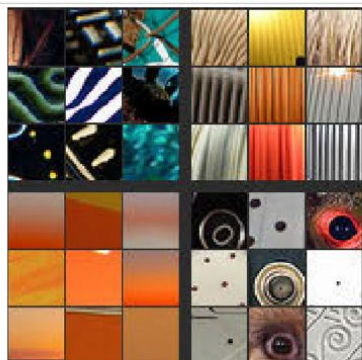
<https://towardsdatascience.com/simple-introduction-to-convolutional-neural-networks-cdf8d3077bac>

Visualizing Trained Filters

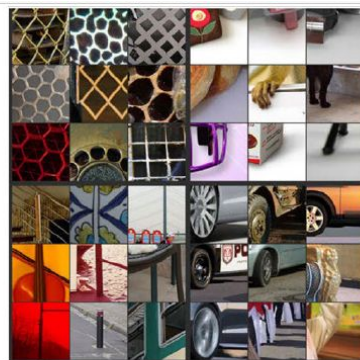
- Optimization techniques can modify input images to maximize activation for a particular class, rather than selecting high-activation images manually.
- Unconstrained optimization of activations can create high-frequency patterns that are difficult to interpret, requiring regularization.



Layer 1



Layer 2



Layer 3

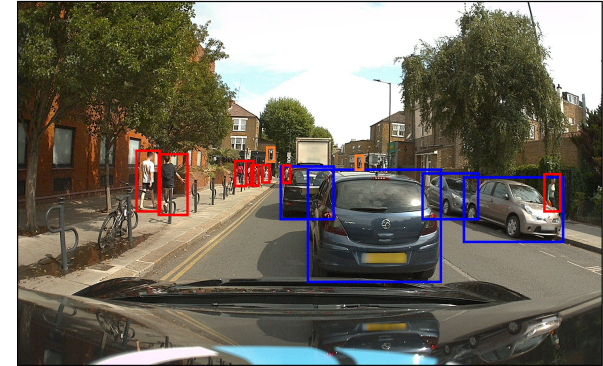


Layer 5

Examples of image patches that produce the strongest activation in the hidden units in a network having five convolutional layers trained on ImageNet data
(Bishop, 2024)

Object Detection

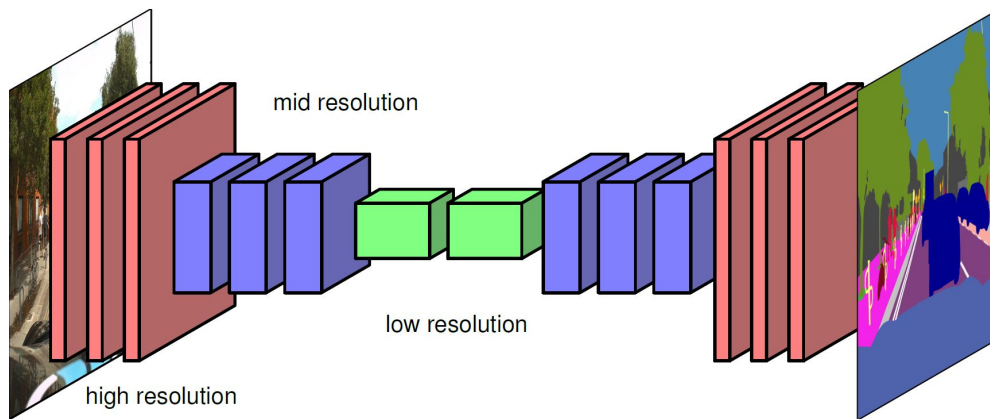
- CNNs are primarily designed for image classification, where an entire image is assigned to a single category (e.g., "cat" or "bicycle").
- CNNs can be adapted for other tasks beyond classification by leveraging their inductive biases, such as spatial hierarchy and feature extraction.
- Many images contain multiple objects belonging to different classes, requiring both detection and classification of each object.
- Object localization is crucial in computer vision applications, such as autonomous vehicles detecting pedestrians, road signs, and other vehicles.



(Bishop, 2024)

Image Segmentation

- Most CNNs use down-sampling (via strided convolutions and pooling) to reduce feature map size, keeping computations manageable while extracting high-level features.
- In semantic segmentation, this down-sampling must be reversed to recover the original image resolution while maintaining learned feature representations.
- Up-sampling techniques serve as the inverse of pooling, increasing the spatial dimensions of feature maps to reconstruct fine-grained segmentation details.



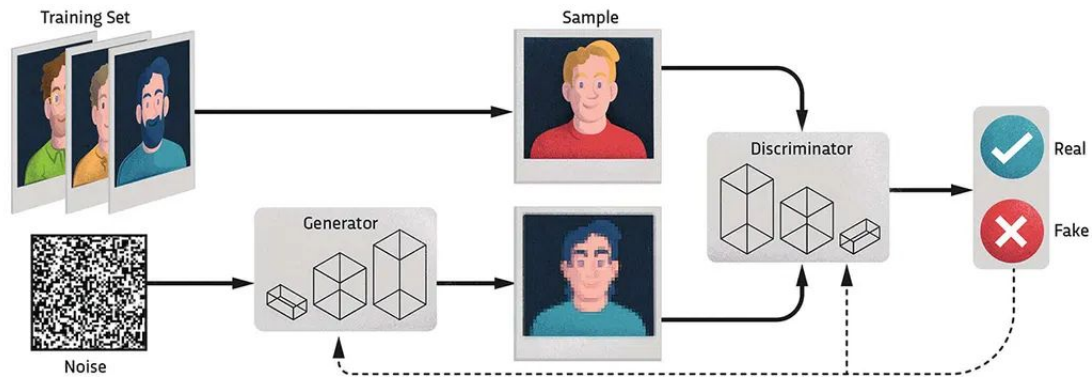
Topics covered

1. Artificial Neural Networks
2. Convolutional Neural Networks
3. **Generative Adversarial Networks**
 - a. Adversarial Training
 - b. Types of GANs
 - c. Applications of GANs
4. Autoencoders
5. Recurrent Neural Networks
6. Attention and Natural Language Models

What are GANs?

- Introduced by Ian Goodfellow in 2014.
- Consist of **two neural networks**: the Generator and the Discriminator.
- Operate in a minimax game: the generator creates data to fool the discriminator, while the discriminator tries to distinguish between real and generated data.
- Widely used for generating realistic images, audio, text, and more.

- Generator (G): Produces synthetic data from random noise.
- Discriminator (D): Evaluates data samples, outputting a probability of authenticity.
- The two networks are trained simultaneously.
- The game is formulated as a minimax optimization problem.



Loss Function - The Discriminator Model

- This loss incentivizes the discriminator to accurately categorize generated samples as fake and real samples with the following equation:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- Where, J_D assesses the discriminator's ability to discern between produced and actual samples.
 - The log likelihood that the discriminator will accurately categorize real data is represented by $\log D(x_i)$
 - The log chance that the discriminator would correctly categorize generated samples as fake is represented by $\log(1 - D(G(z_i)))$
- By minimizing this loss, the discriminator becomes more effective at distinguishing between real and generated samples.

Challenges

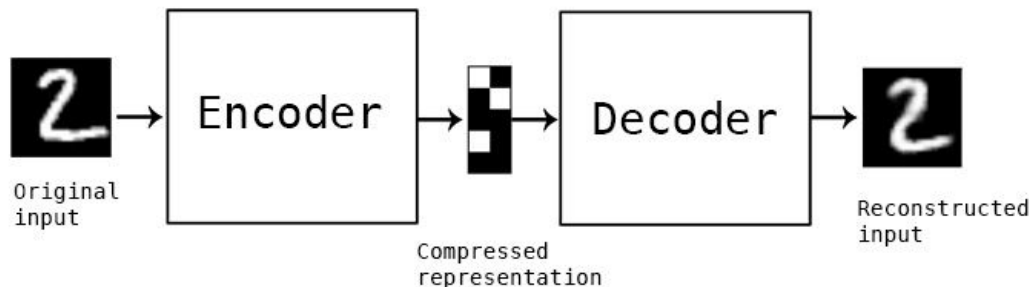
- GAN training can suffer from issues like **mode collapse** (limited diversity in outputs) and **vanishing gradients** (insufficient learning signal), leading to instability.
- **Mitigation Strategies:** Techniques such as feature matching, mini-batch discrimination, and the adoption of alternative loss functions (like the Wasserstein loss) help alleviate these problems.
- **Architectural and Hyperparameter Tuning:** Choosing the right network architectures (for instance, using deep convolutional layers) and carefully tuning hyperparameters are essential for successful GAN training.

Topics covered

1. Artificial Neural Networks
2. Convolutional Neural Networks
3. Generative Adversarial Networks
4. **Autoencoders**
 - a. Deterministic Autoencoders
 - b. Variational Autoencoders
 - c. Applications
5. Recurrent Neural Networks
6. Attention and Natural Language Models

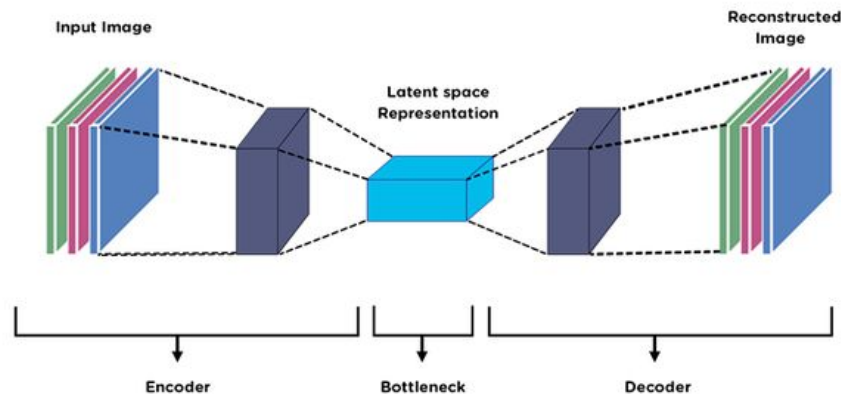
What are Autoencoders?

- Autoencoders are neural networks designed to learn efficient representations of input data → with no need for labels (**Unsupervised**)
 - Also called *Auto-Associative Neural Network*
- They consist of an **encoder** (compresses data) a **decoder** (reconstructs data)
 - The encoder transforms the input data into a reduced-dimensional representation, which is often referred to as “**latent space**” or “encoding”
 - From that representation, a decoder rebuilds the initial input.
- For the network to gain meaningful patterns in data, a process of encoding and decoding facilitates the definition of essential features.



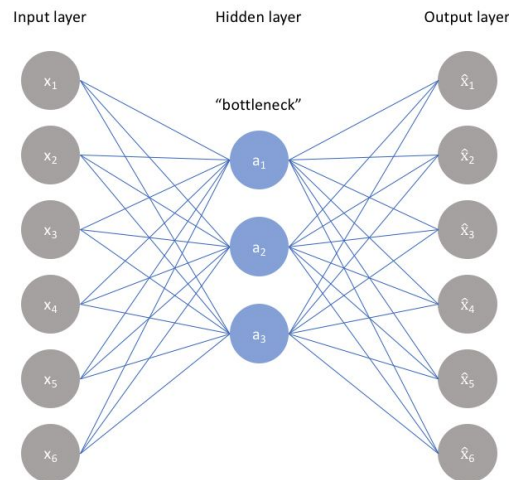
General Architecture of Autoencoders?

- **Encoder:** Compresses input data into a lower-dimensional latent representation.
- **Latent Space (Bottle neck):** Encodes essential features of input data.
- **Decoder:** Reconstructs the original input from the latent space.
- **Loss Function:** Measures reconstruction error (e.g., Mean Squared Error).



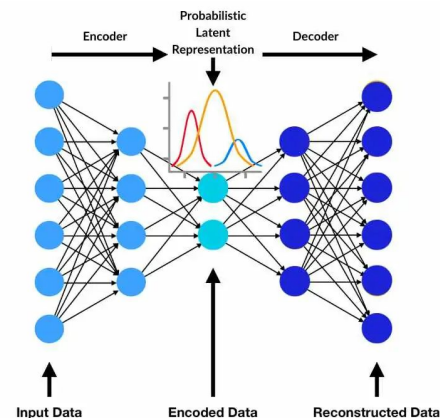
What Makes an Autoencoder Deterministic?

- Encoder maps input deterministically to a single latent representation.
- Uses Mean Squared Error (MSE) or Binary Cross Entropy (BCE) loss.
- No constraints on the structure of latent space.
- Limitation: Cannot generate diverse outputs.
 - Linear Autoencoders
 - Deep Autoencoders
 - Sparse Autoencoders
 - Denoising Autoencoders



Why Variational Autoencoders?

- Introduce a probabilistic approach to learning representations.
- **Encoder:** Compresses input data x into a latent representation z , learning a probability distribution $q(z | x)$
- **Latent Space:** Instead of a single vector, we represent each input as a distribution with mean μ and standard deviation σ
- **Decoder:** Reconstructs the original data from a sample drawn from $q(z | x)$
- Enable sampling of new data points from learned latent space.



Applications of Autoencoders & VAEs

1. Dimension Reduction:

Dimensionality reduction techniques include the use of linear and deep autoencoders.

2. Image Denoising:

For effective and extremely accurate image denoising, autoencoders like the denoising autoencoder can be utilized. Autoencoders, in contrast to conventional denoising techniques, extract the image from the noisy data that has been supplied to them by learning a representation of it. After that, the representation is decompressed to provide a picture without noise. Hence, denoising autoencoders can denoise complicated images that are impossible to denoise using conventional techniques.

Applications of Autoencoders & VAEs

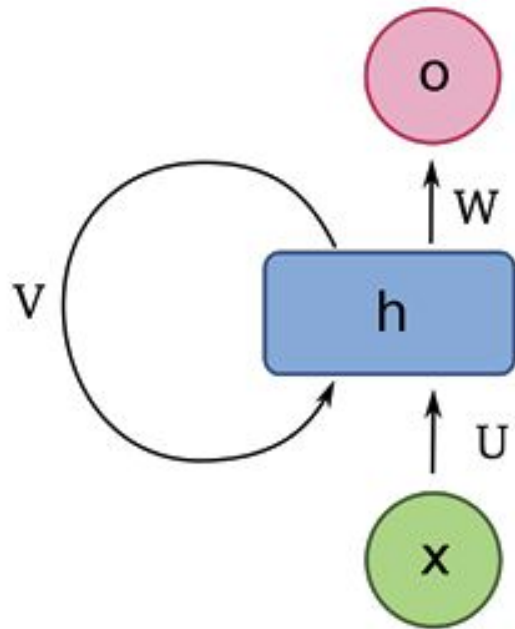
3. Anomaly Detection

Consider an autoencoder that has been trained on a specific dataset P . For any image sampled for the training dataset, the autoencoder is bound to give a low reconstruction loss and is supposed to reconstruct the image as is. For any image which is not present in the training dataset, however, the autoencoder cannot perform the reconstruction, as the latent attributes are not adapted for the specific image that has never been seen by the network. As a result, the outlier image gives off a very high reconstruction loss and can easily be identified as an anomaly with the help of a proper threshold.

Topics covered

1. Artificial Neural Networks
2. Convolutional Neural Networks
3. Generative Adversarial Networks
4. Autoencoders
- 5. Recurrent Neural Networks**
 - a. RNNs - Foundations
 - b. LSTM
 - c. GRU
6. Attention and Natural Language Models

Recurrent Neurons Update



Output Vector

$$Y = O(V \cdot h + C)$$

Update Hidden State

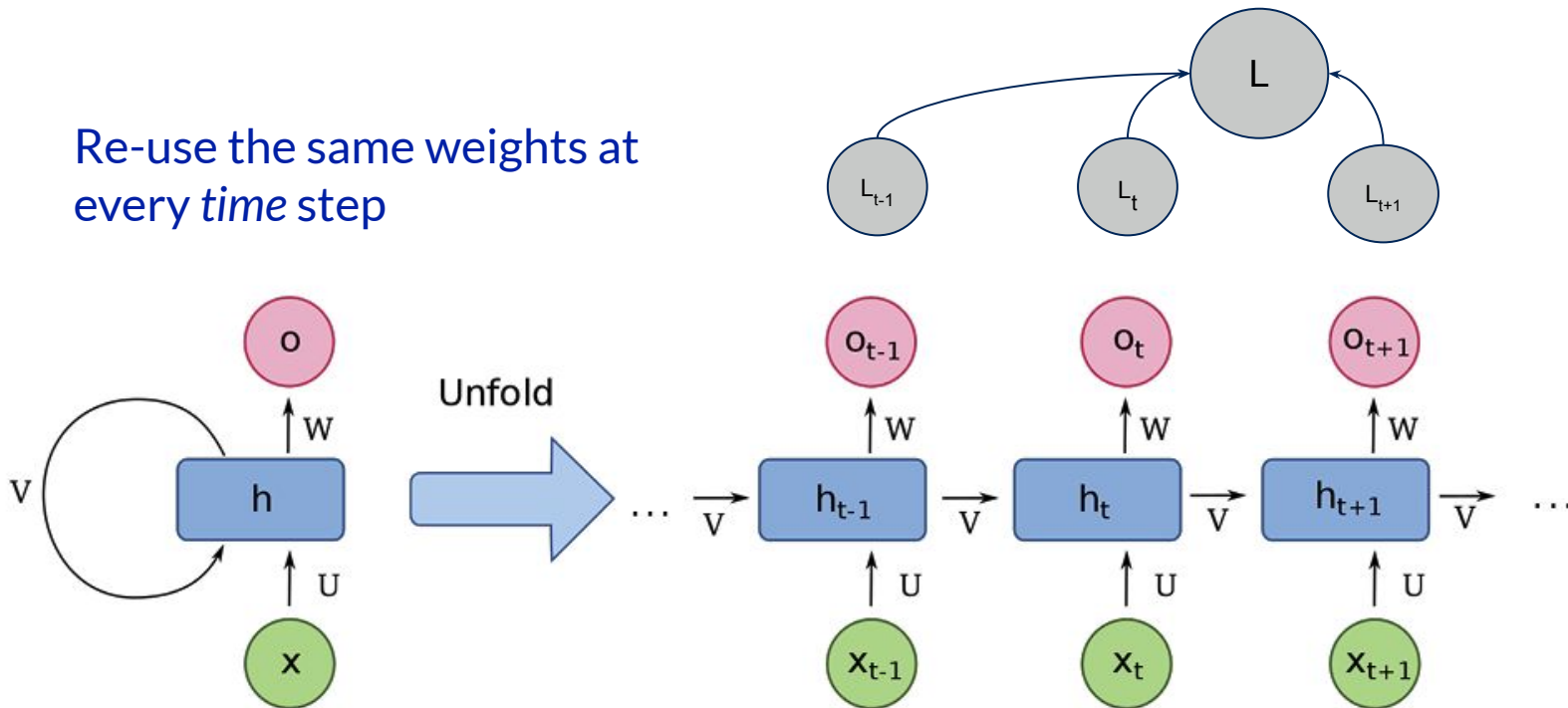
$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Input Vector

$$X_t$$

RNNs unfolding

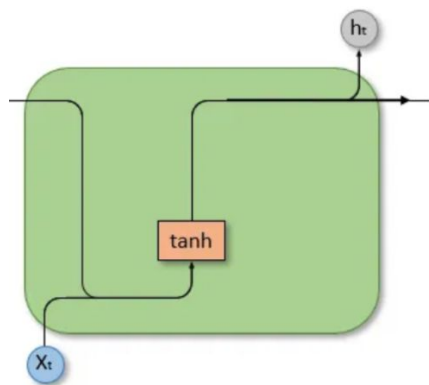
Re-use the same weights at every *time step*



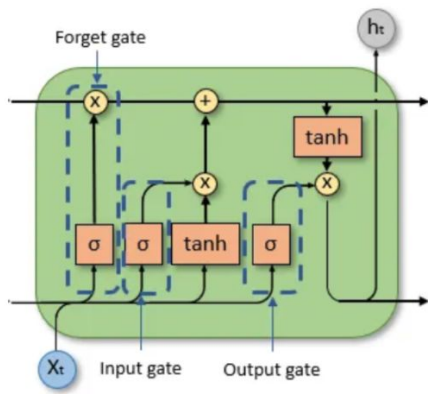
Introduction to LSTMs and GRUs

- LSTMs (Long Short-Term Memory) solve vanishing gradient issues.
- GRUs (Gated Recurrent Units) simplify LSTMs while keeping performance.
- Both use **gating** mechanisms to control memory.

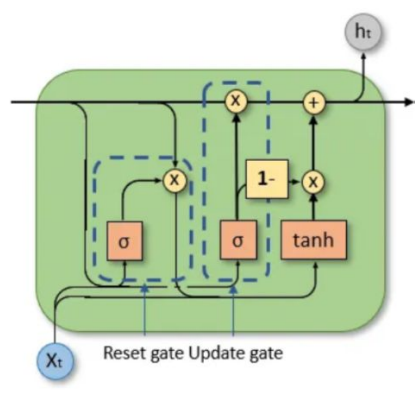
RNN



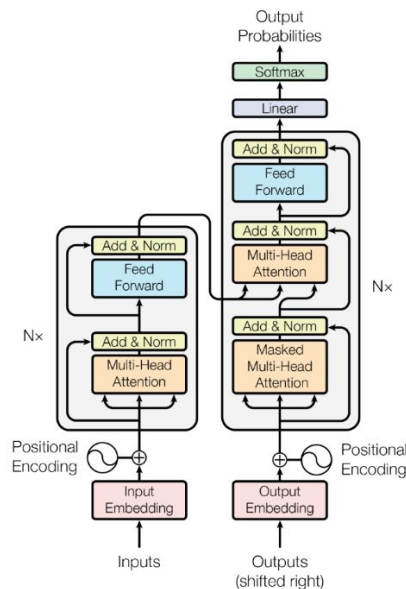
LSTM



GRU



Transformers



<https://aiml.com/compare-the-different-sequence-models-rnn-lstm-gru-and-transformers/>

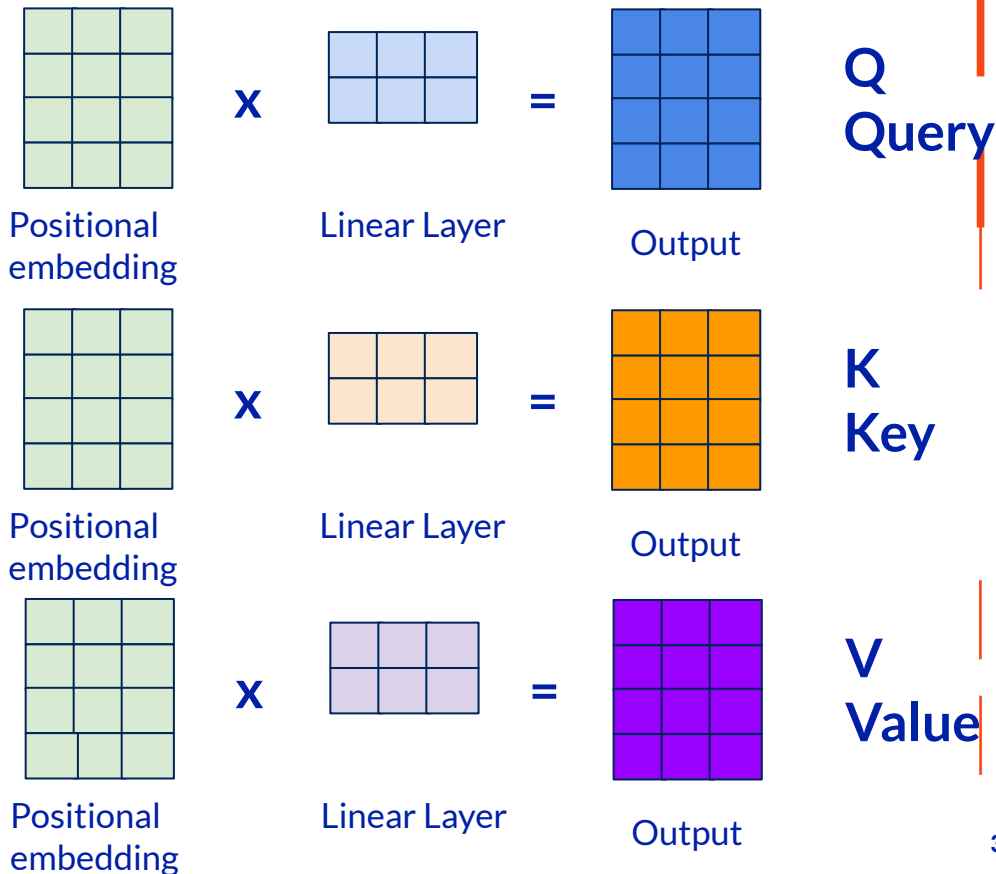
Topics covered

1. Artificial Neural Networks
2. Convolutional Neural Networks
3. Generative Adversarial Networks
4. Autoencoders
5. Recurrent Neural Networks
- 6. Attention and Natural Language Models**
 - a. Attention - Foundations
 - b. Word embedding
 - c. Natural Language Processing Steps

Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search

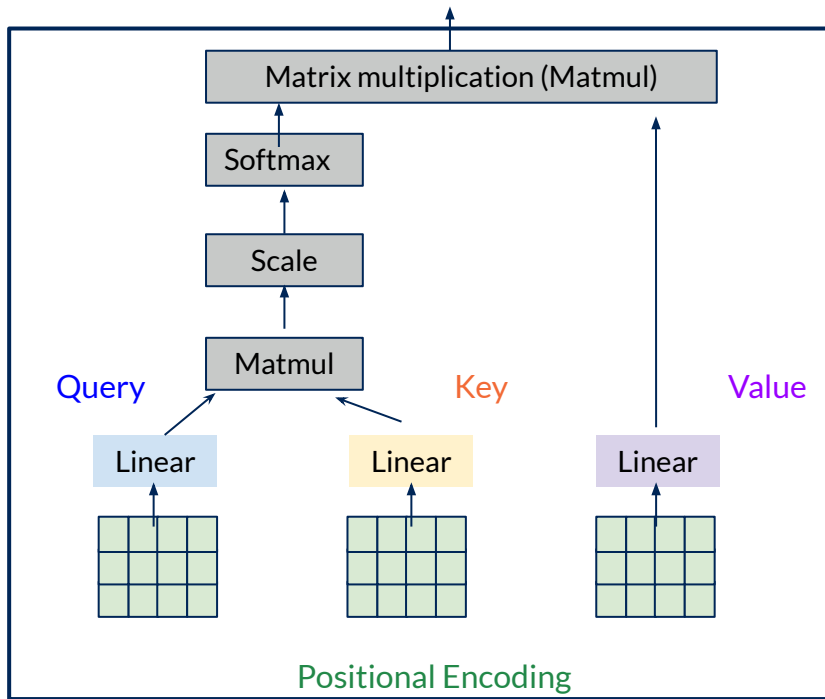


Self- Attention

Goal: Identify and attend to most important features in input X

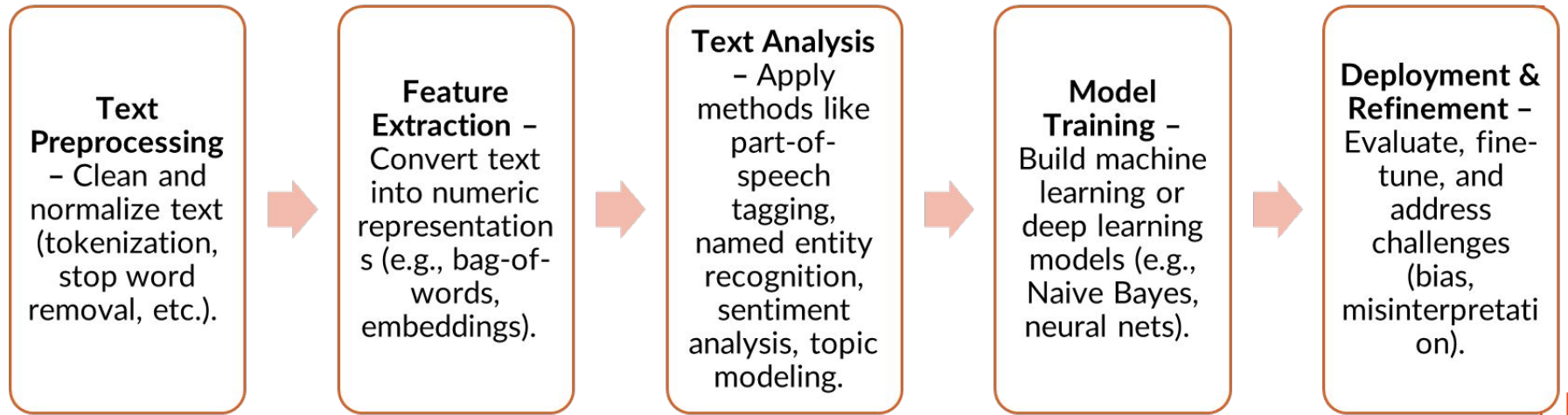
1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

These operations form a self-attention head that can plug into a larger network. Each head attends to a different part of input.



$$\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right) \cdot V$$

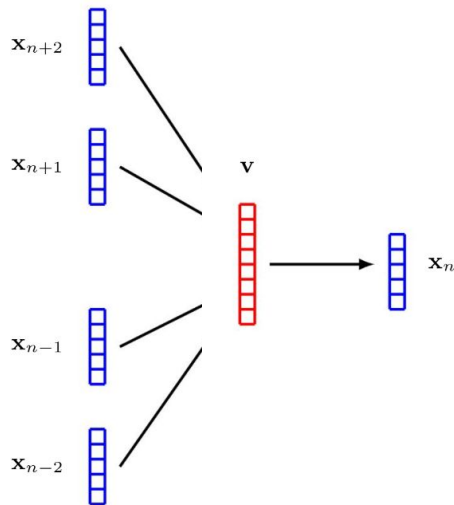
NLP Pipeline Overview



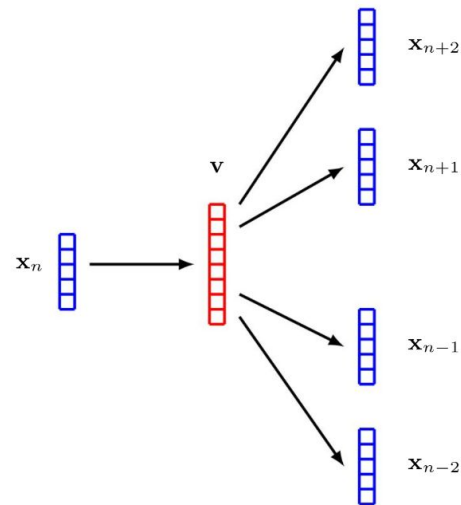
Feature Extraction!! Word Embeddings

- Other Word Embeddings (Word2Vec, GloVe):
 - Dense vectors capturing semantics.
 - CBOW: Predicts center word given context.
 - Skip-Gram: Predicts context given center word.

“The cat **sat** on the mat”



CBOW

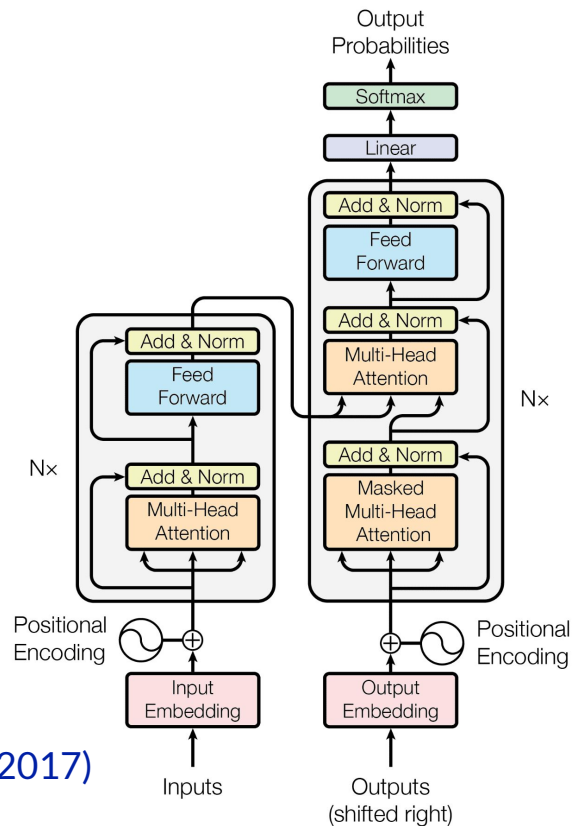


Skip-Grams

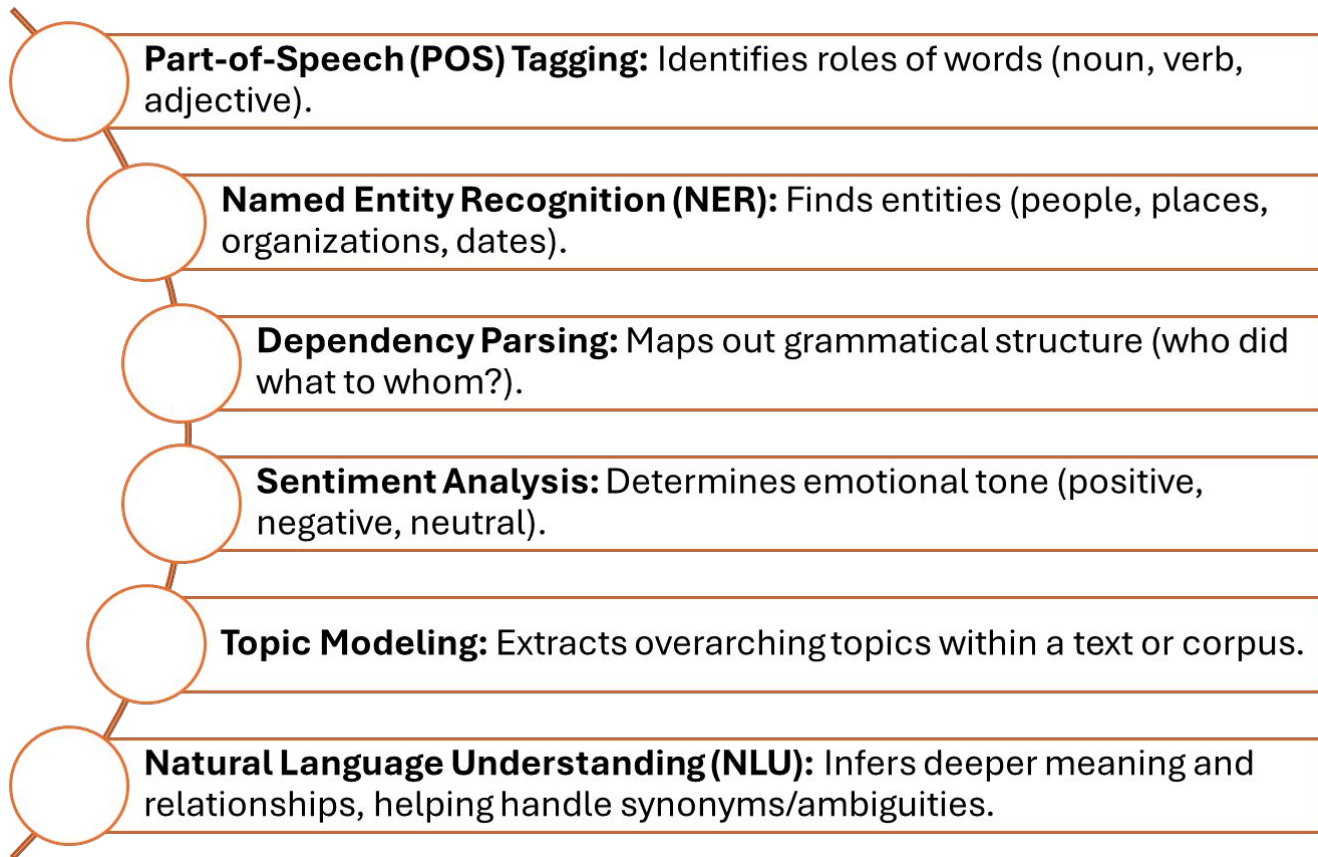
Feature Extraction!! Attention

- Each token embedding “looks at” other tokens.
- Attention assigns weights to decide which tokens are most relevant.
- Output: contextualized embeddings (dynamic, depend on surrounding words).

(Vaswani, 2017)



Text Analysis



Model Training & Inference

1. **Data Preparation:** Preprocessed text → numeric features (e.g., BoW vectors, embeddings).
2. **Train a Model:**
 - a. Classical: Naive Bayes, logistic regression.
 - b. Neural Networks: e.g., a feedforward net with an embedding layer, or a Transformer-based model.
3. **Evaluation & Fine-Tuning:**
 - a. Use validation data to measure accuracy, F1-score, etc.
 - b. Adjust hyperparameters, address errors.
4. **Deployment:** Predict or generate text for new data (e.g., chatbots, translation apps).

Challenges & Risks in NLP

Biased Training

- Using data that under-represents certain groups leads to biased outputs.
- Web-scraped data can inadvertently reflect societal biases.

Misinterpretation

- Speech-to-text errors from dialects, slang, or background noise.
- Garbage in, garbage out (GIGO) problem.

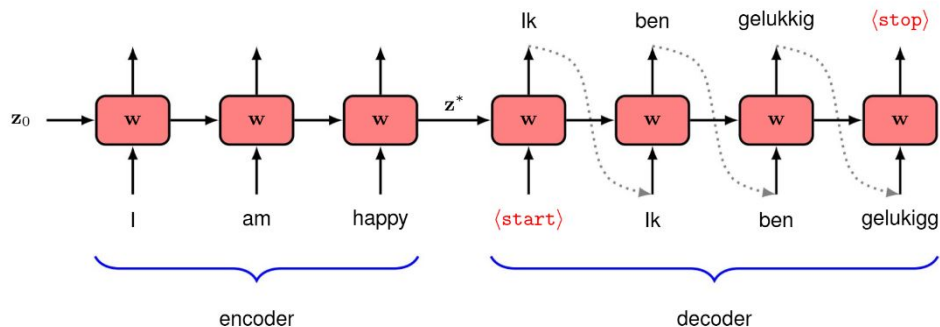
New Vocabulary

- Language is dynamic; new words or slang appear constantly.
- Models must adapt or risk confusion.

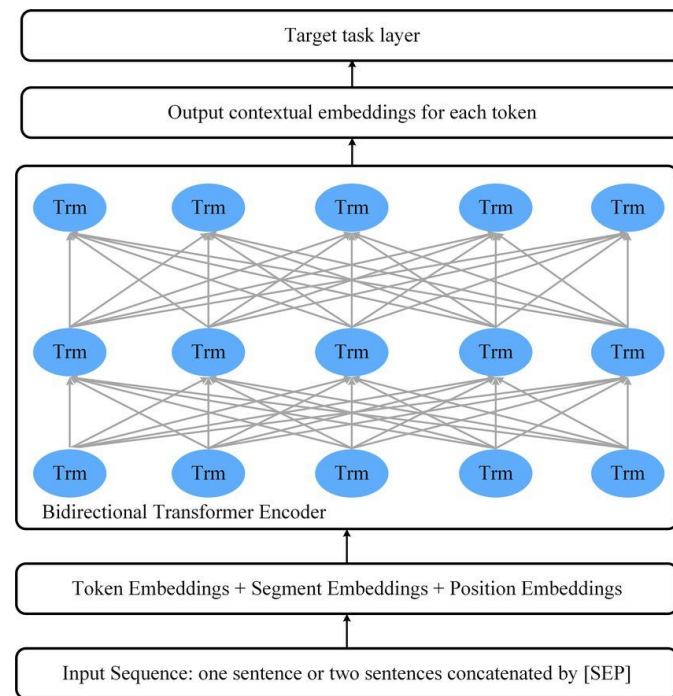
Tone of Voice / Non-verbal Cues

- Sarcasm or emphasis can change meaning.
- Current text-based NLP struggles to fully capture these nuances

Putting things together



Example of an RNN for
language translation (typically
before 2017)
I.e. Google Translate



Example of BERT for search
(2018)
I.e. Google Search

Topics and Labs

1. **Lab 1**
 - Portfolio
2. **Lab 2**
 - Pytorch Fundamentals
3. **Lab 3**
 - CNNs
4. **Lab 4**
 - GANs and Autoencoders
5. **Lab 5**
 - Attention and Natural Language Models



Additional Resources

1. Deep Learning Best Practices (To train deep learning models)
2. RNN - Foundation (Implementation)